

School of Electronic Engineering
and Computer Science

Final Report

Programme of study:
COMPUTER SCIENCE
UNDERGRADUATE

Project Title:
**IMPLEMENTATION OF
A RATING SYSTEM**

Supervisor:
MARC ROTH

Student Name:
SAMUEL LEVY

Final Year
Undergraduate Project 2025/26

Abstract

Rating systems, notably Elo and Glicko, have been used for years. First developed for chess, these systems are based on algorithms that have been implemented in Football, in Go, and in many other competitive settings such as online videogames. This project aims to implement such rating algorithms in a web application adaptable to different game environments. We will provide a website handling the ranking of a playerbase, of user profiles, and allowing users to consult their rating. We will provide an understanding of the algorithms underlying rating systems such as Elo and Glicko, and have a functional implementation of these systems.

Contents

Chapter 1: Introduction	5
1.1 Background	5
1.2 Problem Statement	5
1.3 Aim	6
1.4 Objectives	6
1.5 Report Structure	6
Chapter 2: Literature Review	7
2.1 The Elo system	7
2.2 Glicko (and Glicko-2)	7
2.3 Additional works and experimental feature	8
Chapter 3: Mathematical formulas	9
3.1 Original Elo system	9
3.2 The Glicko system	10
3.3 The Glicko-2 system	10
3.4 Elo/Glicko conversion scale	12
Chapter 4: Application functionalities and design	14
4.1 Preliminary requirements	14
4.1.1 Functional requirements	14
4.1.2 Non-Functional requirements	14
4.2 Application Design	14
4.2.1 Application users	14
4.2.2 System behaviour	15
Chapter 5: Implementation	17
5.1 Database	17
5.1.1 Overview	17
5.1.2 Game table	17
5.1.3 User tables	18
5.1.4 Other tables in the database	20
5.2 Back end	20
5.2.1 Apps	20
5.3 Front end	21
5.3.1 Design	21
5.3.2 Login and Sign up	23
5.3.3 Homepage	24
5.3.4 Search engine	26
5.3.5 Profile page	26
5.3.6 Record Game	28
Chapter 6: Evaluation	29
6.1 Testing rating systems	29
6.1.1 Test results:	30

6.2	User testing	33
6.2.1	User feedback	33
Chapter 7: Conclusions		34
7.1	Achievements	34
7.2	Challenges	34
7.3	Overall Conclusion	34
Appendix C:Gen AI Tools Accountability Statement		37
.1	Database code	40
.2	Rating algorithms model	44
.3	Test code	49
.3.1	Rating algorithms test code	49

Chapter 1: Introduction

Rating systems play a fundamental role in many competitive contexts, ranging from chess to online games. Their purpose is to provide a numerical estimate of each player's level, in order to best reflect their actual performance while allowing for a fair comparison with other participants.

In this project, we are particularly interested in three rating systems: Elo, Glicko and Glicko-2. We seek to understand their behaviour when they are used to rank players in a competitive context.

This project was born from the desire to learn more about and provide an understanding of these algorithms. We will aim to implement a rating algorithm in a web application that could be used for various competitive environments. This application should be simple to use and easy to understand, allowing players to organize competitions with friends.

1.1 Background

Research on rating algorithms can be dated back to Arpad E. Elo now indispensable works on chess. In 1978, Elo developed the homonymous rating system which was quickly adopted by the chess federation [2]. This system, though based on complex probabilities, follows a simple formula. Easy to understand and implement, this algorithm was deemed fairly superior to the old ways of rating players, that mainly relied on players to track their individual progress. This system is now used in many competitive settings, such as Tennis [10] or even video games like *League of Legends* [8]

But the Elo rating system presents some limitations, notably concerning the uncertainty of the rating or the evolution based on participation frequency (more about this later). Because of this, a new system was introduced in 1995 by Mark Glickman, professor in Statistics. This is known as the Glicko (and later Glicko-2) system. This is an update to the Elo rating system aiming to fix its limitations by introducing new parameters that depend on player's history and deeper probabilities. This new system is therefore more complex but more precise. It is currently this system that is used for rankings on websites such as Chess.com.

1.2 Problem Statement

Rating systems are often closed in static environments; if you want to see your chess rating, you have to play chess online, or in competitions, to see your global rating; if you want to have a football rating, you need to be at some kind of professional level; this is applicable to all competitive settings.

This means two things: first, that ratings are often obscure to most users and sometimes create frustration for players that cannot see their improvement; second, that if you ever wanted to simply rate your friends and yourself in a close setting, or participate in smaller scale competitions (10, 20 people instead of worldwide such as chess tournaments), no resource would be available to you to measure your skill against others.

This project aims to solve that issue by implementing a web application of a rating system.

1.3 Aim

The main aim of this project is to provide an understanding of the rating systems underlying competitive games such as chess. It also aims to create a web-application to handle the rankings of competitions. The game would not be handled by such an application; rather, our system would allow users to enter game results and see a real-time update of their rating in the database.

To this aim, both Elo and Glicko ratings would be computed at the same time. Because Elo is easier to understand, but Glicko is more precise, the system would put forward Elo rating for lower-rank players, who would see their rating go up quite fast, allowing them to track their progress, while Glicko would be displayed for higher-rank users. The threshold is still to be determined.

1.4 Objectives

We aim to implement a functional web application (using Django python framework) to handle the rating algorithm. These algorithms will be designed based on A. Elo and M. Glickman research. The application will need to accept two types of users: regular users and admin. We will also need to design and implement a database to store useful information for the system.

If time allows for it, we would also like to try and implement an experimental glicko algorithm variation allowing player history to have a bigger impact on the ranking evolution (more about this in the *Literature review* section).

1.5 Report Structure

This report will be organized in different sections.

In **Chapter 2**, we will do a literature review going over some previous papers; we will explore the underlying algorithms of rating systems, their history, as well as the reason for their evolution.

In **Chapter 3**, we will perform a complete review of the mathematical formulas that will be used for the rating algorithms in our system. This review will be divided in sections based on the algorithms treated in Chapter 2.

In **Chapter 4**, we will explore the application's functionalities and design by reviewing the preliminary requirements and exposing our reasoning and design decisions.

In **Chapter 5**, we will present the implementation of our project, going over the database, the front-end, and the back-end. In **Chapter 6**, we will describe our expected evaluation methods and expose their execution. We will explain the results and information coming from our evaluation methods.

Chapter 7 will constitute a conclusion presenting our achievements and challenges throughout this project.

Chapter 2: Literature Review

Literature review has found that two rating systems were principally two rating systems: Elo [2] and Glicko [3]. These systems both have advantages and inconvenients which are detailed in the papers, and that we will go over here.

2.1 The Elo system

The Elo system is the most well-known and more ancient of the two. It was developed in the 1960's by Hungarian-American chess master and physics professor Arpad E.Elo. According to Elo, it was designed in order to rank players by taking into account several performances rather than one, as *"The obvious purpose of any rating system is to provide a ranking list of whatever is being rated. In a competitive activity such as chess, tournament standings provide tentative rankings, but because individual performances vary from time to time, a ranking list based on a single event is not always reliable."* [2].

This system was more suited to the way chess competitions worked at the time than the systems that were previously used, and it was quickly adopted by FIDE (name given to the World Chess Federation) in 1970. Elo would only publish his works after the system was already democratized, in 1978.

The Elo system is fairly easy to understand, and its probabilistic model is fully based on the two opponents ranking, which makes it adaptable to any competitive setting. The Elo system is currently used in a variety of different games.

However, the Elo system also showed some limitations, such as its inability to take into account the deterioration of skill over time of inactivity, or the inherent randomness that comes at a low level of play. This is why some mathematicians started working on alternatives to Elo's formulas. The one chess uses today is a variation of this developed by Mark Glickman in 1995, called the Glicko system (and its update, the Glicko-2 system).

2.2 Glicko (and Glicko-2)

Developed by Professor of statistics Mark Glickman, the Glicko system is more complex than the original system it is based on. However, it is more precise and aims to alleviate some of the issues of Elo's work. For that purpose, the Glicko scale is different and requires a conversion from and to Elo; it also introduces parameters to account for player's periods of inactivity or skill stability.

Interestingly, this new system also produces asymmetry in the measurements of new ratings. While, in Elo, the outcome of a match generally makes the winner gain x points and the loser lose the same x amount of point, *"in the Glicko system, the amount by which the opponent's rating decreases is governed by both players' RD's"* [3]. Glicko was updated in the 2010's with a new scale called **Glicko-2**. As well as the Rating Deviation of players, it also introduces a value called volatility. This is the scale we will be using in this project when talking about "Glicko".

2.3 Additional works and experimental feature

Despite the added precision of the Glicko system, we felt like something was missing. At the beginning of this project, there was a will to add a parameter to the Glicko scale that would help change the scale based on two opponent's head to head.

The idea was that a player's history could also determine the rating's evolution, as it is possible for a lower-ranked player to beat a higher-ranked player consistently if both competitors play against each other a lot (this is sometimes refer to as "bracket demons" in some competitive fields).

Though this idea seemed interesting, how to implement it without breaking the original scale was a tough maths challenge, and we could not find reliable sources for prior works on the matter. We were able to contact Pr.Glickman to ask for his input and this is what he had to say:

"I don't have much bandwidth to help with this, but your observation that a result may have more information about a player's strength if it's in the context of players whose relative strengths are well-estimated makes sense to me. I think this is a side-benefit of the statistical model that underlies the Glicko-2 system, which you can read about here: <https://www.glicko.net/research/dpcmsv.pdf> - the fully Bayesian model does acknowledge the pairwise covariances in strengths, which is closely related to your proposed idea."

With this e-mail was sent a paper [4] written in 2001 which goes into the probabilistic system underlying the Glicko scale, and seemed to provide inputs for how to implement this new feature.

However, because the maths needed to understand this paper are quite complex, and deeper dive into the subject might divert from the main focus (the system's implementation), it was advised by the supervisor of this project to not go further in the idea and focus on the project itself.

Seeing that this will probably not be implemented, the idea has been disregarded for the project's design; however, if time allows for it, a test feature to experiment with it might be added to the project; some of the project's design is prior groundwork to be able to implement this idea if possible.

Chapter 3: Mathematical formulas

3.1 Original Elo system

Main formula (how a player's score is updated after a game): [2]

$$R' = R + K(S - E)$$

Where:

- R' is the new ranking;
- R is the current ranking;
- S is the score obtained after a game (1 for a win, 0 for a loss, 0.5 for a draw);
- E is the expected score (more about that later);
- K is a parameter controlling the system's sensitivity. A high value makes the score more reactive but potentially less stable.

For this system, we decided to apply FIDE's rating norms and use a K parameter evolving with the player's number of games. This way, a player will experience "placement games" where their rating will change faster, until their skill can be assumed to have been measured correctly. We therefore have:

- If the player is playing their first 8 games, they are in a "placement period" where their skill is measured; their rating will evolve very fast to have an accurate first rating. We have $K = 40$.
- If the player is out of the placement period, and their skill is under a certain threshold of skill, they are in the average player zone; we want their rating to evolve at a reasonable pace. That skill threshold is arbitrary. FIDE puts it at 2400 Elo, and so we will follow that value. In that case, we have $K = 20$.
- Finally, if the player is above the skill threshold, we know that they are very strong and want to measure their skill against other equally skilled players more precise and less reactive. We therefore reduce the K value to $K = 10$.

Expected score:

$$E = \frac{1}{1 + 10^{(R_{\text{opp}} - R)/400}}$$

Where:

- R_{opp} is the rating of the opponent.

If $E > 0.5$, the player is expected to win. If $E < 0.5$, the opponent is expected to win. However, the Elo system presents some limitations. We can fix some of them by using the Glicko system. [6]

3.2 The Glicko system

Glicko introduces uncertainty through the rating deviation RD , and adjusts the update step accordingly. Every player is represented by a rating R , but also by a RD . The lower the RD , the more we can trust the rating.

The rating formulas can therefore be updated [5]:

$$g(RD_{\text{opp}}) = \frac{1}{\sqrt{1 + \frac{3Q^2 RD_{\text{opp}}^2}{\pi^2}}}$$

where $Q = \frac{\ln(10)}{400}$ is a constant.

Glicko also gives this estimation:

$$E = \frac{1}{1 + 10^{-g(RD_{\text{opp}})(R - RD_{\text{opp}})/400}}$$

Given a certain rating period, the player is supposed to face a number m of opponents opp , represented in the indices of the sum. This is because Glicko is intended to work for large “rating periods”, where players would play several games before their ratings get updated.

Since our system aims to update ratings in real time, the rating will not work exactly like that. However, the formulas stay the same; we simply always have $m = 1$. For more simplicity, we decided to keep the formulas as written in Glickman’s works [6].

From these estimations, we can also compute the uncertainty in rating change d^2 and the estimated improvement Δ .

$$d^2 = (Q^2 \sum_{\text{opp}=1}^m g(RD_{\text{opp}})^2 E(1 - E))^{-1}$$

$$\Delta = d^2 Q \sum_{\text{opp}=1}^m g(RD_{\text{opp}})(S - E)$$

These quantities then allow to adjust the rating and the rating deviation:

$$R' = R + \frac{Q}{\frac{1}{RD^2} + \frac{1}{d^2} \sum_{\text{opp}=1}^m g(RD_{\text{opp}})(S - E)}$$

$$\text{and } RD' = \sqrt{\left(\frac{1}{RD^2} + \frac{1}{d^2}\right)^{-1}}$$

Thus, the more a player stays inactive, the more their RD goes up, making their next game more volatile. This allows the system to keep track of the actual level of the player between each game.

But this system still has its flaws, and that’s why we can introduce Glicko-2.

3.3 The Glicko-2 system

Glicko-2 adds the volatility value (σ) [5], which represents the random part of the skill of the player.

The model can be updated. The rating μ , the rating deviation ϕ and the volatility σ can be represented this way:

$$\mu = (r - 1500)/173.7178, \text{ and } \phi = RD/173.7178$$

The g function and the expected value E can then be written:

$$g(\phi_{opp}) = \frac{1}{\sqrt{1 + \frac{3\phi_{opp}^2}{\pi^2}}}$$

$$E = \frac{1}{1 + e^{-g(\phi_{opp})(\mu - \mu_{opp})}}$$

The quantity v is the expected variance of the player's rating based on game outcomes. Δ is the estimated improvement in rating by comparing the pre-period rating to the performance rating based only on game outcomes. E and g are defined as above. m is the number of opponents the player has faced. The opponents' volatilities are not relevant in the calculations.

$$v = \left(\sum_{opp=1}^m g(\phi_{opp})^2 E(1 - E) \right)^{-1}$$

$$\Delta = v \sum_{opp=1}^m g(\phi_{opp})(S - E)$$

Determining the new volatility σ' requires iteration. To compute it, we can follow these steps:

1. Let $a = \ln(\sigma^2)$, and define

$$f(x) = \frac{e^x (\Delta^2 - \phi^2 - \nu - e^x)}{2(\phi^2 + \nu + e^x)^2} - \frac{(x - a)}{\tau^2}.$$

Also, define a convergence tolerance, ε . The value $\varepsilon = 0.000001$ is a sufficiently small choice.

2. Set the initial values of the iterative algorithm.

- Set $A = a = \ln(\sigma^2)$.
- If $\Delta^2 > \phi^2 + \nu$, then set

$$B = \ln(\Delta^2 - \phi^2 - \nu).$$

If $\Delta^2 \leq \phi^2 + \nu$, then perform the following iteration:

- (i) Let $k = 1$.
- (ii) If $f(a - k\tau) < 0$, then

$$\text{Set } k \leftarrow k + 1$$

and go to (ii).

and set $B = a - k\tau$. The values A and B are chosen to bracket $\ln(\sigma^2)$, and the remainder of the algorithm iteratively narrows this bracket.

3. Let $f_A = f(A)$ and $f_B = f(B)$.
4. While $|B - A| > \varepsilon$, carry out the following steps.

(a) Let

$$C = A + (A - B) \frac{f_A}{f_B - f_A},$$

and let $f_C = f(C)$.

- (b) If $f_C f_B \leq 0$, then set $A \leftarrow B$ and $f_A \leftarrow f_B$; otherwise, just set $f_A \leftarrow f_A/2$.
- (c) Set $B \leftarrow C$ and $f_B \leftarrow f_C$.
- (d) Stop if $|B - A| \leq \varepsilon$. Repeat the above three steps otherwise.

5. Once $|B - A| \leq \varepsilon$, set

$$\sigma' \leftarrow e^{A/2}.$$

However, we can compute the new values u' (the rating) and ϕ' (the RD).

$$\phi' = 1 / \sqrt{\frac{1}{\phi^2} + \frac{1}{v}}$$

$$u' = u + \phi'^2 \sum_{opp=1}^m g(\phi_{opp})(S - E)$$

And then, simply convert the RD and rating to the original scale: $r' = 173.7178\mu' + 1500$ and $RD' = 173.7178 \cdot \phi'$

3.4 Elo/Glicko conversion scale

Elo to Glicko

$$u = \frac{r - 1500}{173.7178}$$

Where:

- u is the rating of the player on the glicko scale;
- r is the player's rating on the elo scale.

$$\phi = \frac{RD}{173.7178}$$

Where:

- ϕ is the rating deviation on the Glicko scale;
- RD is the Rating Deviation on the Elo scale.

Glicko back to Elo

$$r' = 173.178\mu' + 1500$$

Where:

- r' is the new rating on the Elo scale;
- μ' is the player's rating on the Glicko scale;

$$RD' = 173.7178 \cdot \phi'$$

Where:

- RD' is the rating deviation on the Elo scale;
- ϕ' is the rating deviation on the Glicko scale;

Chapter 4: Application functionalities and design

4.1 Preliminary requirements

4.1.1 Functional requirements

- **Computations must be accurate:** if rating computations are not accurate, then the whole point of the system is not functional. It is vital to make sure that the Elo and Glicko implementations are correct.
- **Store user information:** storing information such as user Id, ranking, and game history is important for the purpose of this application. Only with this will the users be able to look at their profile.
- **Store updated Elo and Glicko scores:** this is once again the whole purpose of the web application. If computed scores are not stored, ratings will not be updated accordingly, which defeats the purpose of this project. It is important to ensure correct data handling.
- **Allow admin users to change user info and handle games:** this is important for the system to function properly. Whilst we do not want user information to be handled incorrectly by any user, the trusted administrators need to be able to interact with the system. This is the only way competitions or games could take place.

4.1.2 Non-Functional requirements

- **Support growth in number of user and data stored:** The system needs to be able to handle more than just a few users; there is no point in having a rating algorithm if it's just for 5 or 6 players - it would then be really easy to know who is the best, no need for such a system. Therefore our system needs to be able to handle several users.
- **Present ratings and match history to users in a clear and understandable way:** though this will not impact the behaviour of the system greatly, we want users to be able to get information easily. The system needs to convey the key information clearly.

4.2 Application Design

This application follows a design that allows the system to meet its requirements without unnecessary or too complicated features.

4.2.1 Application users

Users of this application can be divided into two roles: *Players* and *Admins*. These two roles each have access to different things, and want to do different things; we will explore what they do and how they do it.

All users share some key features. All users, regardless of their role, are stored in the `USER` table of the database. This table stores their id (which is necessary to identify them uniquely), their username and the password they use to log into the system. This password is naturally stored in a secure manner, as it is hashed by a function available in the Django system.

All users:

- Can log into the system using their name and password.
- Have access to the homepage, which displays a leader board for the players in the system.
- Have access to a search function which allows to look for a player in the database.
- See the profile page of themselves or another player. The profile page shows the name, the id, and the different ratings of a player, and contains information about their win rate (in percentage) and games played.
- Can log out of the system.

Despite all these functions being common, each role still has its specificity. **Players** are the only ones who can play in a game, or have a ranking; however, players themselves do not have access to more features. This is not the case for admin users, who need to have access to advanced functionalities to be able to manage the system.

Admins

- Have access to all the functionalities listed above, as they are considered users.
- Can create games opposing two players, and enter the result. Ratings are updated accordingly.
- Can promote players to admin.
- Can delete users.

4.2.2 System behaviour

This web application is meant to be accessible and easy to use. Therefore, the system has functionalities focused on the core requirements but all of them are useful and easy to use in the scope of the app. As stated above, the system can be used by two types of users: `PLAYERS` and `ADMINS`. `PLAYERS` are the basic users; they have an account with their information, and participate in games, but in the scope of the app, they are simply spectators. A player can see their account or the rating of other players, but cannot alter the system. `ADMINS` are, as the name indicates, administrators of the system. They have access to several functionalities to change user profiles, but most importantly, they are the ones administrating games.

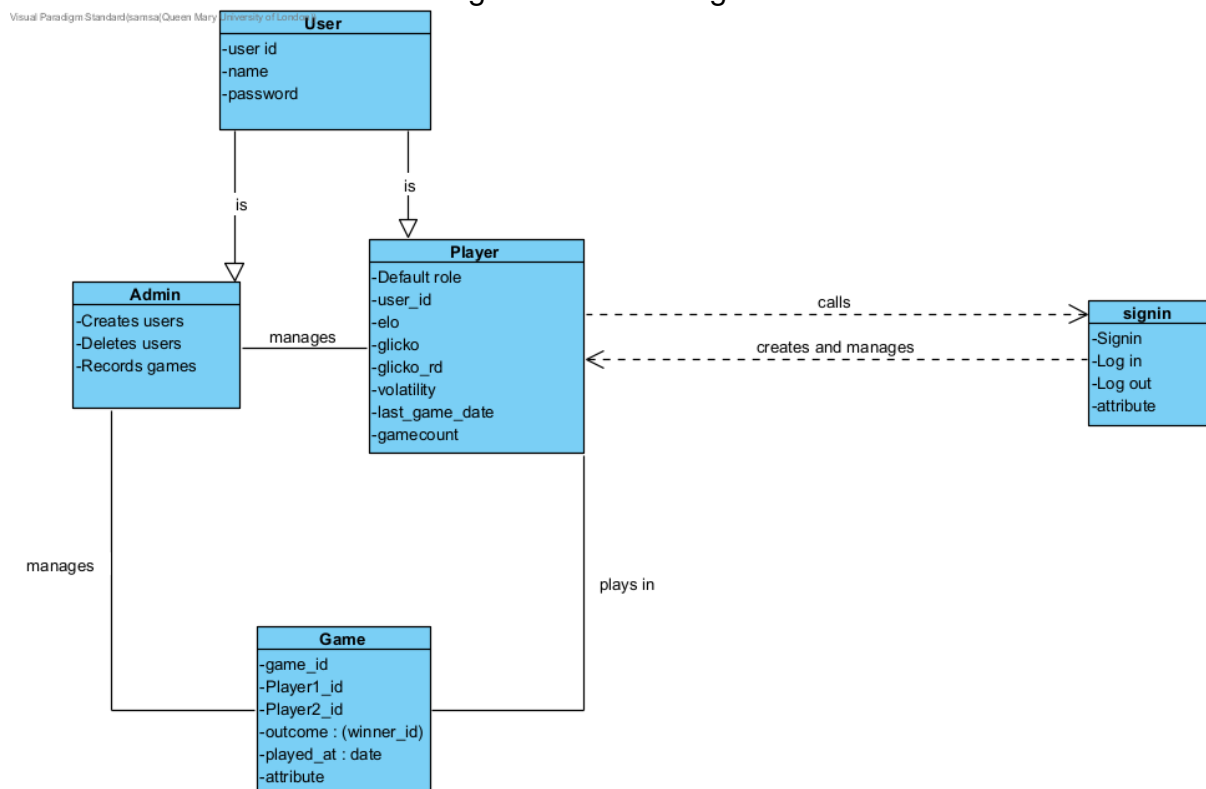
Games are the core of this web application. Only admins can start games and record the results to avoid cheating. Since admins are trusted users and cannot participate in games (at least not from their admin accounts), we assume that they will not use their power to abuse the system and favor some players over others.

The app is divided into 3 web pages, in addition to the log in and sign up menus. Two

of these pages are available to both players and admins; those are the homepage, which displays a leaderboard of players, and the profile page (either the current user or another one, looked up through the search menu), which displays information about a specific player. Some functionalities on the profile page are only available for admins, like promoting a user to admin.

Admin users are also the only ones having access to the last web page, the games one. On that page, admins can fill in a form with both players names and the game result; the results are then recorded, and player ratings are updated in the database. The system can be described with the following diagram:

Figure 1: Class diagram



Chapter 5: Implementation

This project will be implemented using a DJANGO back end, which is a Python framework for web applications [1]. The front end will be programmed using classic web technologies like HTML, CSS and JAVASCRIPT. Finally, the database (coded in SQL) will be handled externally. For the scope of this project, the Database will be handled through XAMPP, a a free software tool written in PHP that is intended to handle the administration of a MySQL or MariaDB database serve [9].

5.1 Database

5.1.1 Overview

The database is done in SQL, and handled externally, outside of the web application. For the purpose of testing, the database is currently managed in XAMPP, but it could be handled in another system if necessary. The SQL code necessary for the implementation of this database is available in the Appendix .1.

This web application requires two main entities and some smaller entities that act as subclasses of the first ones. The two main entities are game and app-user. app-user corresponds to all the possible users of the application and is itself divided in two entities: player and user-admin, for the users that have each role.

Because it is necessary to ensure the proper behavior of our system, we also have django-migration and django-session tables. These tables do not impact the design of the application but they are necessary to enable the website to behave as we want it to.

Table	Action	Rows	Type	Collation	Size	Overhead
☐ admin_user	★ Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ app_user	★ Browse Structure Search Insert Empty Drop	5	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ django_migrations	★ Browse Structure Search Insert Empty Drop	26	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ django_session	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ game	★ Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_general_ci	80.0 KiB	-
☐ player	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	16.0 KiB	-
6 table(s)	Sum	49	InnoDB	utf8mb4_general_ci	176 KiB	0 B

Figure 2: Database overview

5.1.2 Game table

The game entity is crucial to our system, as it represents all the games that are being played between players. This entity's table is made of 5 columns:

1. game-id: the primary key. This is a unique identifier, auto-generated by Django whenever a game is created. This identifier cannot be shared with any other game object of the system. The game-id is a numerical value.
2. player 1: a foreign key column, holding a numerical value. This value corresponds to the ID of a player, in this case the player identified as *Player 1* in the

game object. This value uniquely and unmistakably identifies one of the game's participant.

3. player 2: a foreign key column, holding a numerical value. This value corresponds to the ID of a player, in this case the player identified as *Player 2* in the game object. This value uniquely and unmistakably identifies one of the game's participant.
4. outcome: again, a foreign key column holding a numerical value that identifies one of the participant. When the game object is created, the administrator is supposed to enter a winner for the game. If Player 1 won, *outcome* will hold Player 1's ID; if Player 2 won, *outcome* will hold Player 2's ID; if the game ended in a draw (such situations are possible and likely to happen in games like chess or Tic Tac Toe), then the column accepts a NULL value.
5. played-at: this column holds the date and time at which the game was played. This is not necessary for the proper behaviour of the system, but a player's profile does display their last game's date, and so it is expected to display accurate information.

game_id	player1	player2	outcome	played_at
6	1	2	1	2026-02-17 13:19:41
7	2	1	2	2026-02-17 13:20:30
8	2	1	NULL	2026-02-17 13:44:19
9	1	3	3	2026-02-17 15:40:37
10	2	1	1	2026-02-25 12:25:49

Figure 3: Game table example

5.1.3 User tables

As discussed in Chapter 4, the `user` entity can be divided in two: `player` and `admin`. Both sub-entities share some attributes; these attributes are all stored in the `app-user` table. This table stores the key elements *name*, *ID*, *role*, and *password*, as well as the secondary attribute *date-joined* and some other columns that are set up because of Django but are never put to use. The attributes function as follows:

1. user-id: a numerical, unique identifier generated when the profile is created for the first time. This is the primary key of the table, meaning it is the identifier for each row of the table and is necessary to ensure that the system works properly. It is this attribute we call in `Game` when we call *Player 1*, for example.

2. Name: the username of the user. This is set by the admin when creating the profile, and can be changed from the profile menu. This can be the user's real name or an alias; it is simply a way to identify them.
3. Role: identifies which role the user has been attributed. This attribute can only take two values, ADMIN and PLAYER. This column is used as an indicator for the system to display certain elements to the user depending on the role.
4. password-hash: this column holds the password of the user. This password is stored securely with a hash function integrated with the Django framework. When a password is entered in the *Login* page, it is hashed and compared to the one stored in the database. If they match, the user can be logged in; else, the password is wrong, and the user cannot be logged in. However, this also means that passwords are unreadable from the database.

The other elements of this table are secondary and are either here to ensure that the Django framework works properly or for information purposes.

user_id	name	role	password_hash	date_joined	is_superuser	is_staff	last_login
1	Marc	PLAYER	9950993e3ab28812720d5be16828a9c8b2b99ca9418cc4ecb8...	2026-01-27 17:05:48	0	0	NULL
2	Sorenzz	PLAYER	pbkdf2_sha256\$720000\$ECR9jBhY04g34SjtoGr0Ck\$E0Y8Ab...	2026-01-27 17:05:48	0	0	2026-03-23 11:32:47
3	Dimitrios	PLAYER	9950993e3ab28812720d5be16828a9c8b2b99ca9418cc4ecb8...	2026-01-27 17:05:48	0	0	NULL
4	Admin	ADMIN	pbkdf2_sha256\$720000\$Dc3kmylnlqVTC4BuXAJDA4B\$TwnVfM...	2026-02-01 12:03:11	1	1	2026-03-25 12:14:01
7	Bob	ADMIN	pbkdf2_sha256\$720000\$eYFQFJ1Qu2zXYKBD9vazo\$Gn+7i7...	2026-03-02 12:08:13	0	0	2026-03-04 11:18:08
13	Samuel	PLAYER	pbkdf2_sha256\$720000\$w984UYrIXBMo5NelVRkxMk\$Fem8tZ...	2026-03-16 16:07:37	0	0	2026-03-16 16:07:39

Figure 4: User table example

As previously stated, the user table can then be divided in two entities. The Admin table is simply a list of users with the *admin* role and does not have any other column than the user ID of the people in question. The table itself does not grant any more information or permissions, but it is useful as an indicator to the system that a user has the admin role.

On the other hand, the Player role **requires** a table of its own to handle all of its functionalities. This table is divided into 8 columns:

1. User-id: the primary key, and a foreign key to the User table Id. Acts as a unique identifier for the player in the table and in the database.
2. Elo, and
3. Glicko: rating values for the players in both rating systems, both updated simultaneously after a game has been played. The default value for both rating system is 1500, but the glicko rating tends to noticeably shift more quickly.
4. Glicko-rd: the Rating deviation (*RD*) value for the player. Default value is 350.
5. Volatility: the volatility σ necessary for glicko rating. The base value is 0.06.
6. Last game date: a simple information column containing the date and time of the last game in which the player participated. This information gets displayed on the profile page.

7. Gamecount: a numerical value that displays the number of games the player participated in. The value gets incremented by one every time the user plays. This value is useful to calculate the winrate when showing it on the profile page.
8. Tries: the test value for the test feature mentioned in Section 2.3. At the time of writing this draft report, this value has not yet been implemented, and time schedule indicates that it probably will not be, but it is still a project we would like to explore if possible.

user_id	elo	glicko	glicko_rd	volatility	tries	last_game_date	gamecount
1	1480.000	1389.071	157.101	0.06001	11	2026-03-15 12:19:40	8
2	1500.000	1445.263	239.410	0.06000	6	2026-03-09 17:07:25	3
3	1520.000	1574.873	174.516	0.06000	5	2026-03-15 12:19:40	5

Figure 5: Player table example

5.1.4 Other tables in the database

Though the aforementioned tables are the main ones used in this database, there are a few others that are necessary to our system. The tables *Django-session* and *Django-migrations* are created by Django to ensure its access to the database, and are none of our concern; we will not interact with them at any point in the app.

5.2 Back end

The back end of this web application was developed using the Django framework in Python. This framework allowed to create the web application with 5 classes (see 4.2.2). Each of these classes counts 7 main files. The files *init.py*, *apps.py*, and *tests.py* are created when creating a class, or *app*, in Django; in most cases they will be empty or almost empty, but they are necessary for Django to work properly (except for *tests.py* which can be deleted in the majority of cases).

The back end of this project can be divided into 9 directories, 8 of which are actually intractable and contain code written for this project. Out of these directories, two (*templates* and *static*) handle front end pages; we will talk about them later. 5 other are **apps**.

5.2.1 Apps

Django's official documentation define apps as "a Python package that provides some set of features" [1]. In our project, this corresponds to the classes **game**, **app-user**, **player**, **user-admin**, and **signin**.

- **Game** is the app handling everything related to the `Game` entity of the database; it contains classes and methods to search for players, to record a game into the database, or to update players' ratings after a game.

- **Signin** is the app handling interactions between the system and a user that is not using the system as an `app-user`. This means that it is in this app that you will find classes and methods to sign up to the system (create an account), log in as a user, and log out while on a page.
- **User** is the app handling everything that is relevant for the `app-user` entity, meaning attributes that are common to both admins and players. This includes classes and methods to create users, change user data (i.e change the name of a user), and the implementation of a search engine to look for a specific user.
- **User-admin** is the app handling `admin` actions. In that app we can find methods to create, delete, and alter user objects, as well as a method *makeAdmin* to promote a player to admin status.
- Finally, **Player** handles the `Player` entity, imposing the default values of attributes like the base Elo rating, calculating ratings and *RD* after a game, and allowing to display a player profile in a certain way.

5.3 Front end

Most HTML files are stored in the *templates* directory, and all the associated CSS files are in the *static/css* directory. There is one exception, being the sign in/login webpage, which HTML and CSS files are stored in *signin/templates* and *static*. For the purpose of this document, we will divide the webpages in 2 categories: the common pages and the admin only or admin view pages.

5.3.1 Design

The colour scheme applied over the application is a mix of blue, white, and grey/brown background (the colour code is 736563). We tried to have a consistent interface which colour scheme would make sense across pages.

It was decided to have a common interface between most pages with a header shared between them. We can see on most pages a blue band at the top of the screen with the name of the application, *QMRank*, as well as some functionalities buttons like a search bar (to search for users) or a button to look at the user's profile.

This interface is roughly the same for admin, with the exception of a player profile page since admins do not have a profile; however they can still look up other users. All pages are interconnected with each other, allowing a user to navigate through the application without difficulty.

Let us now take a look at the actual web pages of the application.

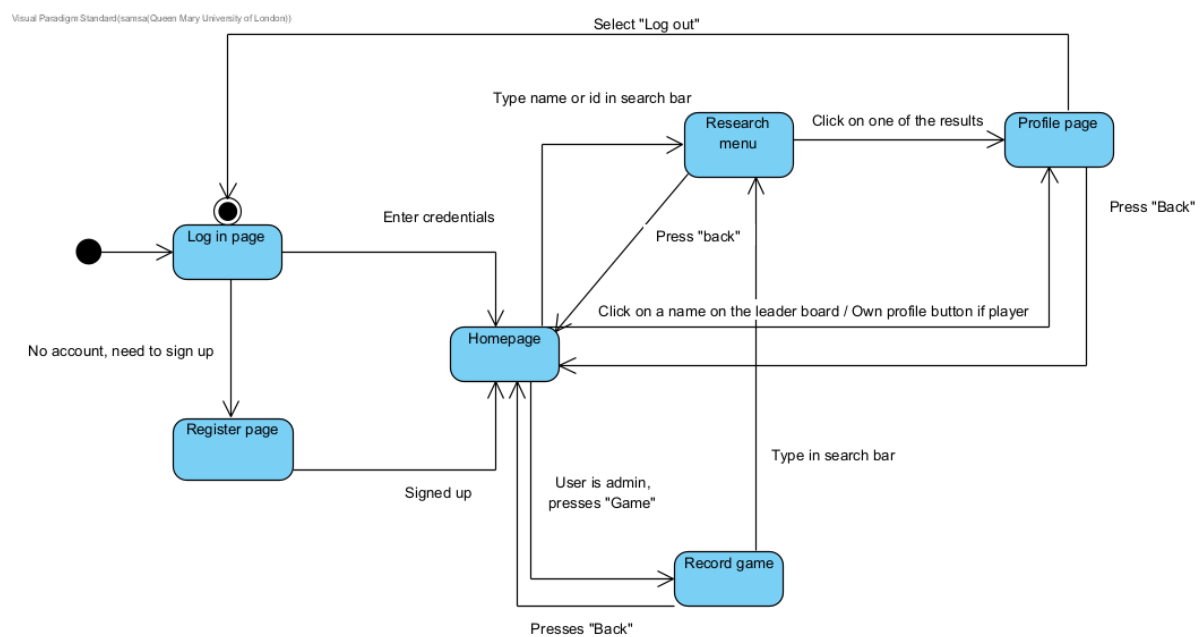


Figure 6: State machine diagram showing app interconnection

Note: This diagram shows the navigation between webpages in the web application. Elements that don't travel to another page are not represented here.

5.3.2 Login and Sign up

Two pages respectively handled in `login.html` and `register.html`, those pages are here to welcome users on the web application. When a user chooses to use the app, they have to log into their account using the login page; if they do not have one, they have to create it using the register page.

The login page is quite straightforward. Handled through a HTML form, the page offers the user two options: to enter their credentials (Username and Password) or to sign up if they do not have an account. What they type password field is of course hidden. When the credentials are entered, the user simply presses the “Log in” button (or presses *Enter* key on their keyboard). If the credentials are found in the database, the user is logged in and brought to the homepage. Else, the page refreshes until they enter correct credentials.

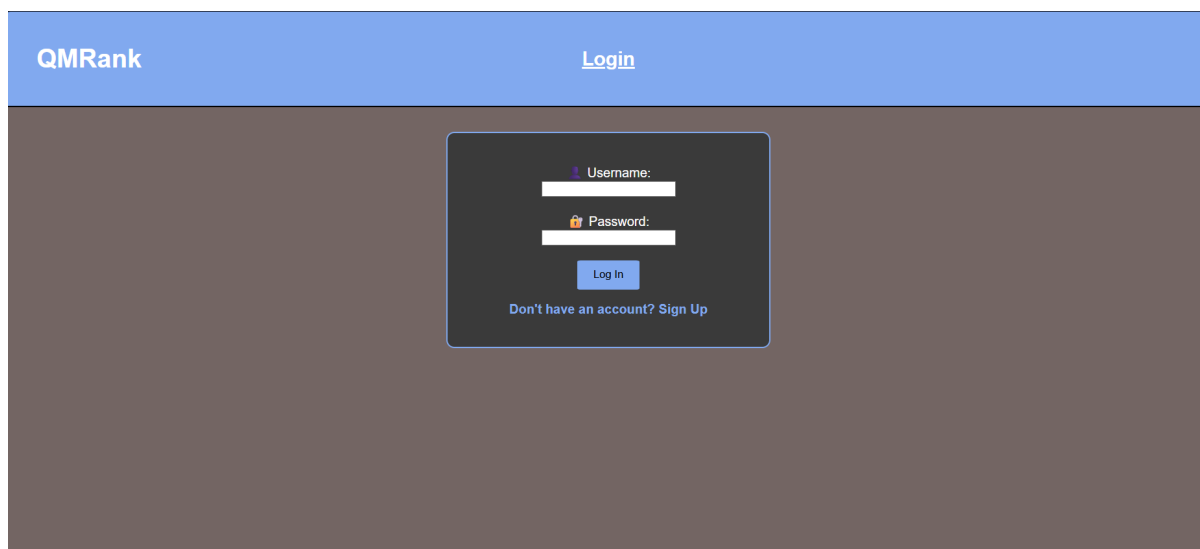


Figure 7: Login page

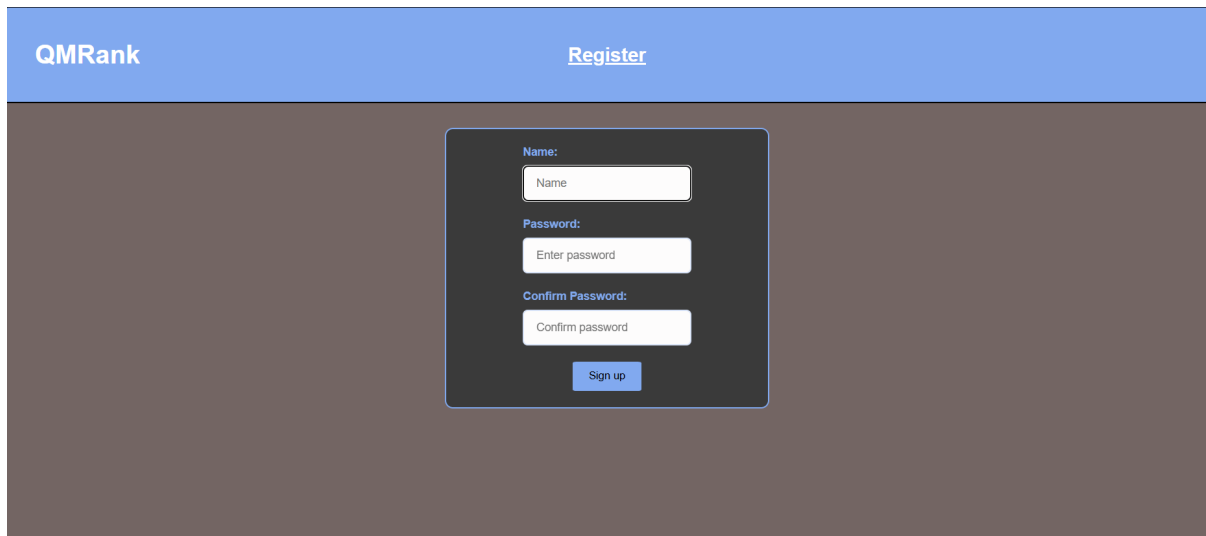
As we can see, the design is simple, and simply aims to deliver the point of the page. But what if the user does not have an account? In that case, they can click on the `Don't have an account? Sign Up` link, which will bring them to the register page.

The register page allows the user to become an actual user, registered into the system's database. It offers three fields to fill in: Name, Password, and Password again (the two passwords have to match, as a common security measure to make sure the user did not make an error typing it in). When these fields are complete, the user can click the “Sign up” button to create their account.

The system handles several possible errors that can happen on that page: the password has to be 8 characters or more, the two fields have to match, some passwords are considered insecure...

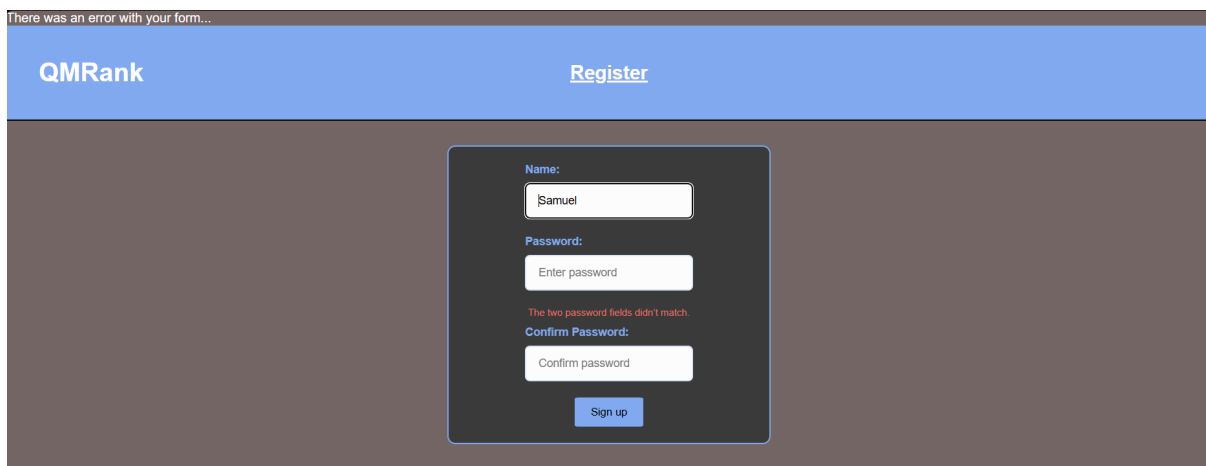
If these criteria are not respected, the system will warn the user with a red text explaining the error above the field in question, but also a small error message at the top of the screen, signifying that the sign up failed.

If log in or registration is successful, the user is taken to the homepage.



The screenshot shows the 'QMRank' website's registration page. The header is blue with 'QMRank' on the left and 'Register' on the right. The main content area is dark grey. In the center is a white registration form with the following fields: 'Name:' with a text input containing 'Name', 'Password:' with a text input containing 'Enter password', and 'Confirm Password:' with a text input containing 'Confirm password'. Below these fields is a blue 'Sign up' button.

Figure 8: Sign up page



This screenshot shows the same registration page as Figure 8, but with an error. At the top, a red message bar says 'There was an error with your form...'. The 'Name' field now contains 'Samuel'. The 'Password' field contains 'Enter password'. The 'Confirm Password' field contains 'Confirm password'. A red error message, 'The two password fields didn't match.', is displayed between the password and confirm password fields. The 'Sign up' button remains at the bottom.

Figure 9: An error example: the passwords do not match

5.3.3 Homepage

Whether the user is a player or an admin, the homepage will display a leader board of the highest ranked players in the system. These players are displayed with their rank, name, and rating. The leader board is configured to only show the top 20 players registered in the system. Each player name can act as a link to their player profile page (more about this later). The page will also display a search bar that allows the user to look for player profiles, by ID or by name.

The Player homepage displays these two options as well as a profile button showing the name of the current logged in user. Pressing this button will take the user to their own profile page.

The admin homepage also displays the leader board, but instead of a simple search bar, it offers a menu with two options: "Search" will reveal the hidden search bar and allow the admin to look for a profile. But most importantly, the "Game" option will take the admin to the Game recording page.

The screenshot shows the Player homepage. At the top, there is a blue header bar. On the left, a circular profile picture placeholder contains the name 'Samuel'. In the center, the text 'QMRank' is displayed. On the right, there is a search bar with the placeholder text 'Search players...' and a 'Search' button with a magnifying glass icon. Below the header, the main content area has a dark gray background. It features a crown icon followed by the title 'Leaderboard'. Underneath the title is a table with three columns: 'Rank', 'Player Name', and 'Elo'. The table contains four rows of data.

Rank	Player Name	Elo
1	Dimitrios	1520
2	Samuel	1500
3	Sorenzz	1500
4	Marc	1480

Figure 10: Player homepage

The screenshot shows the Admin homepage. The top blue header bar contains the 'QMRank' logo on the left and three navigation buttons on the right: 'Leaderboard' (with a crown icon), 'Search' (with a magnifying glass icon), and 'Start Game' (with a game controller icon). The main content area has a dark gray background. It features a crown icon followed by the title 'Leaderboard'. Underneath the title is a table with three columns: 'Rank', 'Player Name', and 'Elo'. The table contains four rows of data.

Rank	Player Name	Elo
1	Dimitrios	1520
2	Samuel	1500
3	Sorenzz	1500
4	Marc	1480

Figure 11: Admin homepage

5.3.4 Search engine

The search engine supports research based on username or user ID. After typing the research in the search bar, users are taken to a list of all players corresponding to what was typed in. For example, searching user Id “1” would display players with ID 1, 10, 11, etc. and looking for a user by their name would display them and any other player who would share the username or something similar (for example, looking for “Marc” would display “Marc” and “Marcus”).

From the search menu, the user is given the opportunity to display another user’s profile. They can also do another research by typing into the search bar, or go back to the home page by clicking on the “Go Back” option.

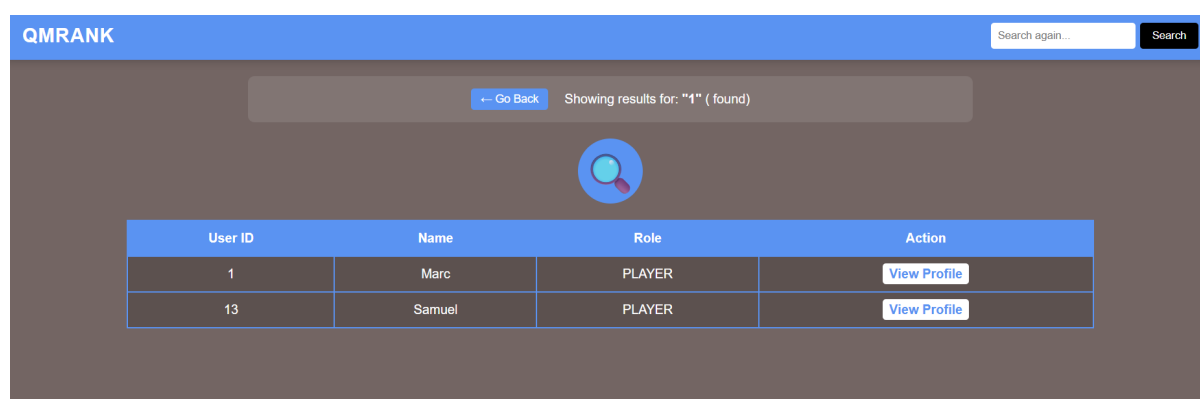


Figure 12: Search results menu

5.3.5 Profile page

Clicking on the “View Profile” option will bring the user to a page displaying the profile of the player they wanted to see. This is the same page to which clicking on a name in the leader board or when clicking on the profile button in the homepage. The profile page is a bit different depending on if the user is looking at another player’s profile, their own profile, or if an admin is looking at a profile. Here are the possibilities:

- **What the player profile actually displays** is information about the player that the user is examining. This includes name, user id, rank in the leader board and rating, as well as some additional information such as the number of games played, the player’s win rate (meaning the percentage of times they won out of their total number of games, value that is calculated by browsing through the game table) and the date of last game played.
- **If a player is looking at another player’s profile**, the page also offers them the ability to make another research through the search bar, look at their own profile by selecting the round profile button on the top left, going back to the home page, or log out of the system (which will bring them to the log in page).
- **If a player is looking at their own profile**, the page is basically the same, with a new option: changing their username. This can be done by clicking on the option and then typing it into the small menu that will pop up. Changing an username in the web application instantly changes it in the database. User ids cannot be modified.

- If an admin user is looking at a player profile, they are offered two more options than a regular user. They can *delete* a user or *promote them to admin*. Deleting a player deletes all records of the player, meaning their entry in the Player and in the User table get deleted. The game they participated in stay saved, as they are needed for other players records. Promoting a user to admin means deleting them from the Player table (as the two roles are mutually exclusive to ensure fairness) and creating an entry for them in the admin-user table. Their user entry stays saved.



Figure 13: Search results menu



Figure 14: Own profile page

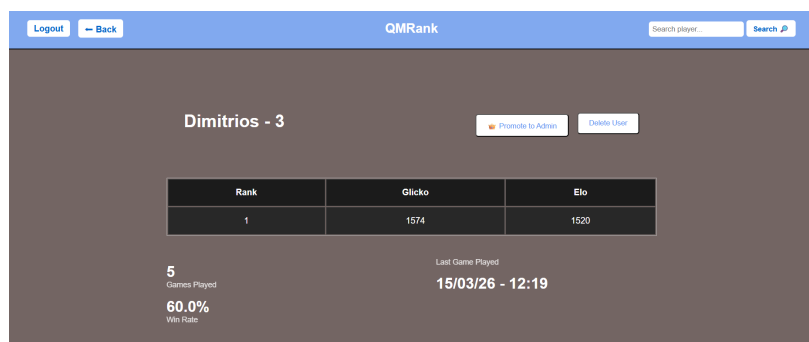


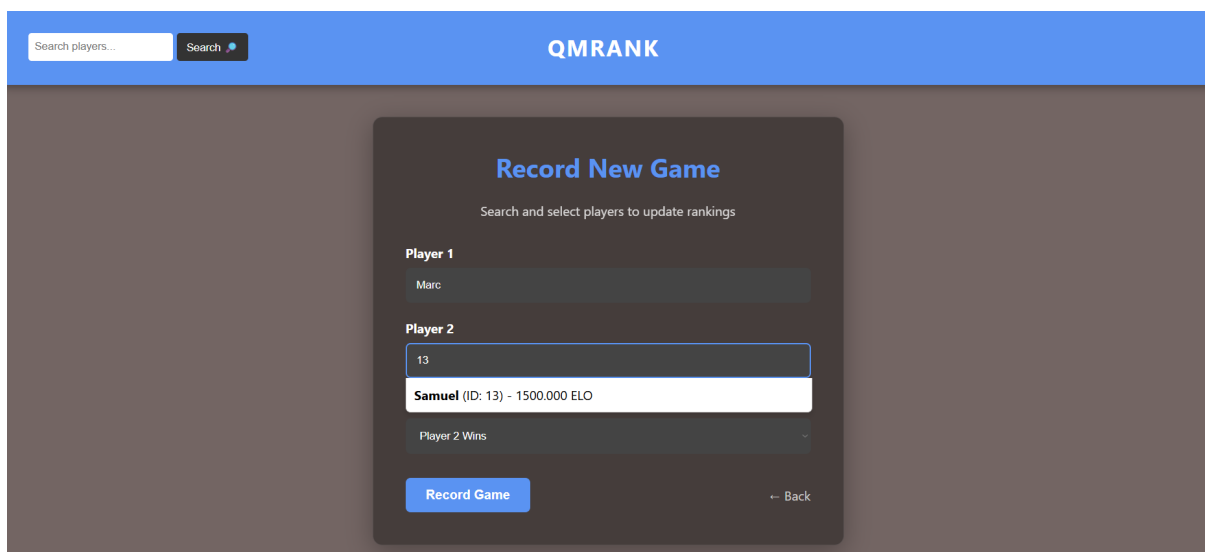
Figure 15: Admin view of profile page

Readers might have noticed that the admin view of the page does not allow for a “View your own profile” option. This is because admin do not have a user profile; that option is reserved for players.

5.3.6 Record Game

Only available to admin users, this option is nonetheless the key feature of our system. The *Record Game* page is where the magic happens; when a game is recorded, players' ratings are updated, as well as their *RD* and their volatility. This is page is the core of our application. It is handled in the *games.html* file, and calls a HTML form to post the update request to the database.

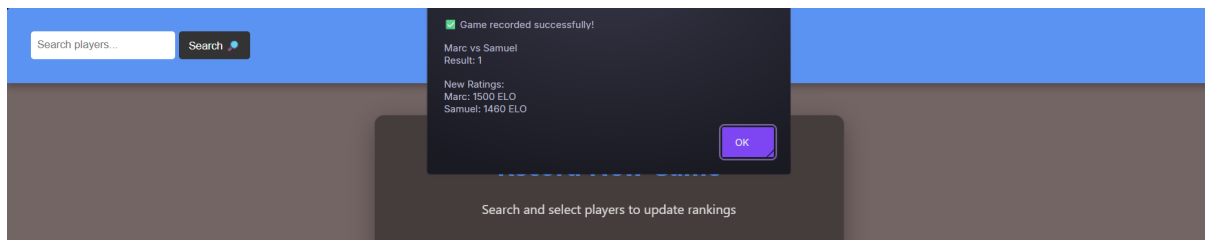
When on the *Record Game* page, the admin user is expected to enter both players and the result. To enter players, they only have to type user Id or username and select the corresponding player in the rolling menu that appears. This rolling menu only appears after 2 characters minimum have been entered, so players with a user ID below 2 characters will not show up on the menu (they will if it is their username that is looked up). Once the players are entered, the admin can select the result between 3 options: "Player 1 wins", "Player 2 wins", and "Draw".



The screenshot shows the 'Record New Game' interface. At the top, there's a blue header with the 'QMRANK' logo and a search bar. Below this, a dark gray modal box contains the form. The form has a title 'Record New Game' and a subtitle 'Search and select players to update rankings'. It includes two player selection fields: 'Player 1' with the value 'Marc' and 'Player 2' with the value '13'. Below the 'Player 2' field, a dropdown menu shows 'Samuel (ID: 13) - 1500.000 ELO'. A dropdown for the result is set to 'Player 2 Wins'. At the bottom of the modal, there is a blue 'Record Game' button and a 'Back' link.

Figure 16: Record game page with ID search

There are two ways to get out of this page: the button "Back" brings the user back to the home page, while the search bar will bring them to the search result menu. As soon as the results are entered, they are registered in the database and a pop up message displays the new calculated ratings. The ratings are updated in the database and will be shown as updated on the leader board and user profiles.



This screenshot shows the same 'Record New Game' modal box as Figure 16, but with a success message overlay. The message is in a dark gray box with a green checkmark icon and the text 'Game recorded successfully!'. It details the match: 'Marc vs Samuel', 'Result: 1', and the updated ratings: 'New Ratings: Marc: 1500 ELO, Samuel: 1460 ELO'. An 'OK' button is located at the bottom right of the message box.

Figure 17: Message from recorded game

Chapter 6: Evaluation

The evaluation method will consist of the classical steps carried out in order to evaluate such a project. The application will need testing, which will be carried out by myself as well as some other test users to ensure no bias in the testing process. This testing will include:

- Test the robustness of the ratings by testing different match settings: testing the different game outputs is necessary.
- Testing user functionalities by creating several profiles
- User testing to see if regular users can understand the UI.

6.1 Testing rating systems

To test the robustness of the rating system, we decided that the best evaluation method was to run test games on test users outside of the system. This choice was made so test cases could be automatized, allowing for rapid testing rather than having to manually test all entries through the web application. For that purpose, we designed a testing script available in .3 in the Appendix. This code checks that the rating algorithms are functional and robust by testing 7 separate issues:

1. **Rating evolution of two equally matched players alternating wins and losses:** with two players of equal skill playing with each other, and having an equal amount of wins and losses against one and another, their ratings should stay close to the default values (1500) while still changing since the first few games will have more impact than the later ones due to the K parameter. The testing script therefore create two players and records 20 games (10 wins each, 10 loss each). If the end ratings are close but not equal to 1500, we know our rating algorithms work well in that case.
2. **A dominant player always beats a weaker one:** we are now testing that our system updates correctly when a player dominates another one. We create two users and make them play against one another for 20 games. The result we should observe is that the dominant player's rating should increase and the weaker player's decrease. Rating evolution should stay slow as the dominant player is always expected to win. The results are entered manually, as shown in .3. The goal of this test is simply to observe the behaviour of the rating algorithms and ensure that they function as expected.
3. **A weak player beats a stronger player:** we are now testing our upset factor, to see what happens when a significantly weaker player beats an established player. What we expect to happen is the stronger player's rating to drop significantly as they keep losing games they should have won. Similarly to test case 2, the results are entered manually; they are simply here so we can observe the behaviour of the ratings.

4. **Testing RD 's evolution:** we are simulating 10 games and observing the evolution of a player's RD . What we expect to see is the RD slowly decreasing as a player plays more and more games, becoming more consistent.
5. **Testing the K factor:** the K factor for Elo is supposed to change depending on the number of games played and the player's rating. We simply verify that this is the case by simulating 20 games. The first 8 (placement games) should on average change the Elo of a player more than the later games. We compare the average values and check if they match our expectations.
6. **Testing a known skill ordering when several games are played:** we establish a skill order of 4 players from strongest to weakest. We then make them play 100 games with each other. The results are randomized, with the expected winner (and so the respective rating of the players) being used as a bias for the result. We expect our final ratings to match the actual and objective skill rating that we have established earlier. If they do, then we know our rating algorithm measures skill adequately.
7. **Test scalability:** to test if our rating algorithms would still be trustworthy with a large number of players and games, we create 20 player objects, ordered by "objective skill" and compute them playing randomly against one another for 500 matches, following the same method as for the previous test. At the end of this scenario, we expect that the final ratings will match our skill ordering, that there will be a significant gap between higher skilled and lower skilled players, and that the values will update at a reasonable rate without drifting too much from expected values.

6.1.1 Test results:

Figure 18 to 20 show the test results for our script. Our test results are satisfying, showing the robustness of our system and its adaptability to different situations and variable scale. Our tests all produced the expected results, showing that our ratings behave as we want them to, and can therefore be used as a reliable skill measure.

Results breakdown:

1. The first test shows, for the two players: an Elo score of 1494 and 1505, and a Glicko score of 1488 and 1511. Our evaluation stays close to 1500 (our default value), as it should. We can deem the test successful.
2. The second test gives the stronger player an Elo and Glicko score of 1654 and 1889 respectively, and the weaker player scores of 1345 and 1110 respectively. This is the expected behaviour with the dominant player being rated higher but his rating not going too high, so we can put this test as successful.
3. The weaker player beating a stronger player gives us an Elo drop of 38.77 and a Glicko drop of 52.92. For a single game, this is quite a high value, which is the upset factor we expected to see. The test is successful.

4. In our test of RD 's evolution, we can see it going from 350.0 to 128.8 in 10 games. This is a significant yet progressive decrease which is in the expected range. The test is successful.
5. We can see that with a K factor of 40, the average Elo change is of 15.09, while it is only 4.89 with a lower k . Therefore, the K value is deemed impactful and our test is a success.
6. We can see that our ranking order matches the expected skill ordering (Best-Good-Average-Weak). We can therefore deem the test as successful.
7. Our final scalability test shows that for 20 players and 500 matches:
 - games are computed quickly (500 matches in 0.0693 seconds),
 - Glicko and Elo ordering matches "objective" skill ordering (Elite - Strong - Average - Below Average - Beginner),
 - there is a clear distinction between the different levels of play (181.67 Elo points between Elite and Beginner and 248.35 Glicko points),
 - and the Elo and Glicko ratings behave as expected.

Therefore, we can deem this final test successful.

```
=====
TEST 1: Two equal players, alternating wins
=====
  Alice | Elo: 1494.14 | Glicko: 1488.60 | RD: 89.51 | Vol: 0.060019 | W/L: 10/10
  Bob   | Elo: 1505.86 | Glicko: 1511.40 | RD: 89.51 | Vol: 0.060019 | W/L: 10/10
PASSED: Equal players stay near 1500

=====
TEST 2: Player A always beats Player B
=====
  Strong | Elo: 1654.38 | Glicko: 1889.32 | RD: 181.70 | Vol: 0.059993 | W/L: 20/0
  Weak   | Elo: 1345.62 | Glicko: 1110.68 | RD: 181.70 | Vol: 0.059993 | W/L: 0/20
PASSED: Dominant player rated higher

=====
TEST 3: Low-rated player beats high-rated player
=====
  Favorite | Elo: 1761.23 | Glicko: 1747.08 | RD: 100.02 | Vol: 0.060009 | W/L: 0/1
  Underdog | Elo: 1238.77 | Glicko: 1252.92 | RD: 100.02 | Vol: 0.060009 | W/L: 1/0
Favorite Elo drop: 38.77
Favorite Glicko drop: 52.92
PASSED: Upset causes meaningful rating change

=====
TEST 4: Rating Deviation decreases over games
=====
  RD progression: 350.0 → 290.3 → 260.5 → 228.1 → 204.5 → 184.7 → 169.0 → 156.0 → 145.3 → 136.4 → 128.8
PASSED: RD decreases with more games

=====
TEST 6: K-factor is higher during placement (first 8 games)
=====
  Avg Elo change during placement (k=40): 15.09
  Avg Elo change after placement (k=20): 4.89
PASSED: K-factor correctly higher during placement
```

Figure 18: Test results 1 to 5

```

=====
TEST 7: Tournament with known skill ordering
=====
Final standings (should be Best > Good > Average > Weak):
  Best | Elo: 1587.61 | Glicko: 1546.82 | RD: 70.73 | Vol: 0.060000 | W/L: 36/16
  Good | Elo: 1567.17 | Glicko: 1508.86 | RD: 69.98 | Vol: 0.060039 | W/L: 32/21
  Average | Elo: 1512.00 | Glicko: 1442.34 | RD: 68.92 | Vol: 0.060012 | W/L: 24/27
  Weak | Elo: 1333.64 | Glicko: 1178.33 | RD: 81.65 | Vol: 0.059994 | W/L: 8/36
✓ PASSED: Elo ranking matches true skill order
✓ PASSED: Glicko ranking matches true skill order

```

Figure 19: Test 6 results

```

Expected tier order: ['Elite', 'Strong', 'Average', 'BelowAvg', 'Beginner']
Elo tier order:      ['Elite', 'Strong', 'Average', 'BelowAvg', 'Beginner']
Glicko tier order:   ['Elite', 'Strong', 'Average', 'BelowAvg', 'Beginner']

Elite vs Beginner Elo separation: 181.67
Elite vs Beginner Glicko separation: 248.35

Total Elo: 29924.89 (expected 30000), drift: 75.1086

WITHIN-TIER ELO VARIANCE:
  Elite: std_dev = 21.68
  Strong: std_dev = 61.04
  Average: std_dev = 50.92
  BelowAvg: std_dev = 30.86
  Beginner: std_dev = 26.16

RANK CORRELATION WITH TRUE SKILL (Spearman):
Elo: 0.8962 (1.0 = perfect)
Glicko: 0.8887 (1.0 = perfect)

PERFORMANCE:
500 matches computed in 0.0693 seconds
7216 matches/second

ASSERTIONS:
✓ Elo tier ordering matches true skill
✓ Glicko tier ordering matches true skill
✓ Elo separation between Elite and Beginner > 100 (181.67)
✓ Glicko separation between Elite and Beginner > 100 (248.35)
✓ Elo drift within expected range for K-factor asymmetry (75.11 < 480)
✓ Elo rank correlation > 0.7 (0.8962)
✓ Glicko rank correlation > 0.7 (0.8887)
✓ Performance OK: 500 matches in 0.0693s

✓ TEST 9 PASSED: Scalability test complete

```

Figure 20: Test results for scalability

6.2 User testing

User testing was conducted by having a small group of users navigating the system, observing their behaviour, and ask for their feedback. The experiment was conducted in two parts. First, the users were simply asked to create an account (using the *sign up* function) and navigate the website, experimenting with the features and discovering elements of the design. They were then logged in as admin users and were asked to record a game and its results between specific users, as well as to update some user details.

Users seemed to understand the system pretty well, not running into any significant issue. The testing sessions did allow to discover some bugs in the system (such as an issue in the score displays, which was fixed afterwards), but nothing impacting the user experience overall.

6.2.1 User feedback

User feedback was overall pretty positive, showing that the system was functional, simple to use and easy to understand. However, some criticisms were made of some elements of the design. Notably, some users complained about the use of pop-up messages for some feedback elements, such as the updated scores after a game or the message indicating a successful sign up to the system. These users asked for more feedback screens rather than simple alerts, making the experience smoother and easier to follow.

Chapter 7: Conclusions

7.1 Achievements

We have discovered and learned about the two main rating systems for games: Elo and Glicko. We went into the maths underlying both algorithms, as well as the expected functionalities of our future web application. We displayed the functionalities and the design of the web application, showing database schema and screen pictures of the app. This system shows functioning rating algorithms and an understandable interface.

The objectives of the project were achieved, and the problem statement answered. This project has delivered a functioning, tested, and usable rating system web application; our literature review and our exploration of rating systems themselves provided us with an understanding of the underlying algorithms that govern competitions.

From a technical and personal perspective, this project was very instructive. It provided me with experience and understanding of the technical elements used, such as the Django framework, Python, HTML, CSS, JavaScript and SQL; it showed the importance of database design and allowed me to dive deeper into database technologies; finally, it allowed to put into practical use lessons learnt throughout the Computer Science course for the purpose of a greater project.

7.2 Challenges

With more time, the web application could be expanded to respond to some issues. For example, the system is currently only designed for desktop screen and could use a CSS update to support smaller screens such as phones. Similarly, issues raised by users during the testing could be handled, such as improving user feedback or the design of the messages implemented to transmit message to the web application's user. In another category, more time allocated to this project would have allowed us to explore the experimental feature described at the beginning of the report. The current system architecture is designed with this feature in mind, which could allow research without interfering with the correct behaviour of the web app too much. Deeper research into the matter was not possible due to the limited resources, time frame and mathematical knowledge; however, such an addition would have made the web application a more interesting piece of software, and failing to implement it is a shortcoming.

7.3 Overall Conclusion

Beyond the technical achievements and challenges of this project, this work highlights the relevance of rating systems in competitive environments. Rating systems such as Elo, Glicko, Glicko-2 but also algorithms that were not treated here such as the Harkness or the UKD system, mostly play a social role, providing players with measurable, objective reflections of their skill level, encouraging competition. Implementing such systems from scratch has demonstrated that even a small-scale web application can replicate the core mechanics used by large platforms such as Chess.com or competitive gaming services.

The building of this project also highlighted the importance of system design: decisions made early in the project on the database schema structure, the separation of

user roles, or even which rating algorithms to use had lasting consequences on what was or was not achievable within the project's time frame. This reflects a wider truth in software engineering, that architecture and planning are just as important as the code itself.

Overall, this project has delivered a functioning, tested, and usable rating system web application. It demonstrates that implementing a reliable skill measurement tool is possible without the resources of a large organisation, and that open mathematical frameworks like Glicko provide a solid, well-founded basis on which to build.

References

- [1] Django Software Foundation. Applications — Django documentation. <https://docs.djangoproject.com/en/6.0/ref/applications/>, 2024. Accessed: 2024-05-20.
- [2] Arpad E. Elo. The Rating of Chess Players, Past and Present. Arco Publishing, 1978.
- [3] Mark E. Glickman. The glicko system. Technical Report 16, Boston University, 1995.
- [4] Mark E. Glickman. Dynamic paired comparison models with stochastic variances. Journal of Applied Statistics, 28(6):673–689, 2001.
- [5] Mark E. Glickman. Example of the glicko-2 system. Technical Report 28, Boston University, 2012.
- [6] Mark E. Glickman. The glicko-2 rating system. Technical report, Boston University, 2013. Online technical document.
- [7] Ryan Kirkman. pyglicko2: Python implementation of the glicko-2 rating system. <https://github.com/ryankirkman/pyglicko2>, 2013. GitHub repository, accessed: 2026-03-30.
- [8] League of Legends Wiki Contributors. Elo rating system, 2024. Accessed: 2026-04-20.
- [9] phpMyAdmin Development Team. phpmyadmin documentation: Introduction, 2026. Accessed: 2026-03-30.
- [10] Tennis Abstract. Atp elo ratings, 2026. Accessed: 2026-04-20.

Appendix A: Gen AI Tools Accountability Statement

ECS635U – Project Module
Generative AI Tools – Student Accountability Statement

The completed and signed statement must be included in the appendix of the progress and final reports.

Student Name: _Samuel Levy_____

Project Title: Implementation of a rating system_____

Supervisor Name: Marc Roth_____

1. Have you used Generative AI tools for your project?

YES

If yes, please complete sections 2 and 3 and sign the declaration below.

If no, please sign the declaration below.

2. Please provide details of all the AI tools that you used in your project work.

AI Tool	Version	Publisher	Access URL
e.g. ChatGPT	3.5	OpenAI	https://chat.openai.com/
Claude Opus	4.6	Anthropic	https://claude.ai/login

3. If you have used Artificial Intelligence (AI) tools in any part of your project please include in table format how, where, and why these tools were used. This should include references to the relevant parts of your project work — for example:

- Specific chapters or sections of the report (e.g. literature review, methodology, requirements)
- Software development (e.g. AI-assisted code generation, debugging, documentation)
- Interview or survey materials (e.g. AI-generated or refined questions/prompts)
- Datasets (e.g. AI-assisted data cleaning, labelling, or synthesis)
- The showcase video (e.g. AI-assisted scripting, narration, or editing support)

Be specific and transparent about your use of AI tools, including both direct outputs (e.g. text, code, images) and indirect uses (e.g. idea generation, editing assistance). The goal is to ensure academic integrity and provide a clear record of how AI contributed to your work.

Example Table – please add your own

Project Section	Description	Justification
e.g. Literature Review	Used AI to generate some LaTeX script	I used GenAI to generate some LaTeX script when mine was facing bugs. All the text has been written by my hand only.
Testing script	Used AI to generate test cases for the rating tests	Testing could be very slow otherwise, so I used AI to generate test cases faster. I then personally analyzed the results.

3. Generative AI Student Declaration

- I have declared all uses of AI tools (e.g., coding assistants, text generators, data analysis tools) in this project.
- I have reviewed and verified all AI-assisted content to ensure accuracy and suitability.
- I take full responsibility for the originality and integrity of my project submission.
- I have referenced all sources, including those suggested or produced with AI support.
- I have not used AI in ways that undermine academic honesty (e.g., plagiarism, unverified code submission).
- My use of AI was supplementary and did not replace my own independent learning and problem-solving.

Signed: Samuel Levy_____ Date: 30/03/2026__

Appendix B - Code implementation

.1 Database code

This is the SQL code currently used for the database design. This database is handled externally but is constantly interacted with by the system.

```
--
-- Table structure for table 'admin_user'
--

CREATE TABLE 'admin_user' (
  'user_id' int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=
  utf8mb4_general_ci;

--
-- RELATIONSHIPS FOR TABLE 'admin_user':
--   'user_id'
--     'app_user' -> 'user_id'
--

--
-- Triggers 'admin_user'
--
DELIMITER $$
CREATE TRIGGER 'trg_admin_role_chk' BEFORE INSERT ON 'admin_user'
  FOR EACH ROW BEGIN
  DECLARE v_role VARCHAR(10);

  SELECT role INTO v_role
  FROM app_user
  WHERE user_id = NEW.user_id;

  IF v_role <> 'ADMIN' THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'User must have role ADMIN to appear in
        ADMIN_USER.';
  END IF;
END
$$
DELIMITER ;

-- -----
--
-- Table structure for table 'app_user'
--

CREATE TABLE 'app_user' (
  'user_id' int(11) NOT NULL,
```



```

    'name' varchar(100) NOT NULL,
    'role' varchar(10) NOT NULL,
    'password_hash' varchar(255) NOT NULL,
    'is_superuser' tinyint(1) DEFAULT 0,
    'is_staff' tinyint(1) DEFAULT 0,
    'date_joined' datetime DEFAULT current_timestamp(),
    'last_login' datetime DEFAULT NULL,
    'is_active' tinyint(1) NOT NULL DEFAULT 1
) ;

--
-- RELATIONSHIPS FOR TABLE 'app_user':
--
-- -----
--
-- Table structure for table 'django_migrations'
--

CREATE TABLE 'django_migrations' (
  'id' int(11) NOT NULL,
  'app' varchar(255) NOT NULL,
  'name' varchar(255) NOT NULL,
  'applied' datetime(6) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=
  utf8mb4_general_ci;

--
-- RELATIONSHIPS FOR TABLE 'django_migrations':
--
-- -----
--
-- Table structure for table 'django_session'
--

CREATE TABLE 'django_session' (
  'session_key' varchar(40) NOT NULL,
  'session_data' longtext NOT NULL,
  'expire_date' datetime NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=
  utf8mb4_general_ci;

--
-- RELATIONSHIPS FOR TABLE 'django_session':
--
-- -----

```

```

--
-- Table structure for table 'game'
--

CREATE TABLE 'game' (
  'game_id' int(11) NOT NULL,
  'player1' int(11) NOT NULL,
  'player2' int(11) NOT NULL,
  'outcome' int(11) DEFAULT NULL,
  'played_at' datetime NOT NULL DEFAULT current_timestamp()
) ;

--
-- RELATIONSHIPS FOR TABLE 'game':
--   'outcome'
--     'player' -> 'user_id'
--   'player1'
--     'player' -> 'user_id'
--   'player2'
--     'player' -> 'user_id'
--
-----

--
-- Table structure for table 'player'
--

CREATE TABLE 'player' (
  'user_id' int(11) NOT NULL,
  'elo' decimal(10,3) DEFAULT NULL,
  'glicko' decimal(10,3) DEFAULT NULL,
  'glicko_rd' decimal(10,3) NOT NULL DEFAULT 350.000,
  'volatility' decimal(10,5) NOT NULL DEFAULT 0.06000,
  'tries' int(11) NOT NULL DEFAULT 0,
  'last_game_date' datetime DEFAULT NULL,
  'gamecount' bigint(20) NOT NULL DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=
  utf8mb4_general_ci;

--
-- RELATIONSHIPS FOR TABLE 'player':
--   'user_id'
--     'app_user' -> 'user_id'
--
--
-- Indexes for dumped tables
--

```

```
-- Indexes for table 'admin_user'
--
ALTER TABLE 'admin_user'
  ADD PRIMARY KEY ('user_id');

--
-- Indexes for table 'app_user'
--
ALTER TABLE 'app_user'
  ADD PRIMARY KEY ('user_id');

--
-- Indexes for table 'django_migrations'
--
ALTER TABLE 'django_migrations'
  ADD PRIMARY KEY ('id');

--
-- Indexes for table 'django_session'
--
ALTER TABLE 'django_session'
  ADD PRIMARY KEY ('session_key'),
  ADD KEY 'django_session_expire_date_idx' ('expire_date');

--
-- Indexes for table 'game'
--
ALTER TABLE 'game'
  ADD PRIMARY KEY ('game_id'),
  ADD KEY 'idx_game_p1' ('player1'),
  ADD KEY 'idx_game_p2' ('player2'),
  ADD KEY 'idx_game_outcome' ('outcome'),
  ADD KEY 'idx_game_playedat' ('played_at');

--
-- Indexes for table 'player'
--
ALTER TABLE 'player'
  ADD PRIMARY KEY ('user_id');

--
-- AUTO_INCREMENT for dumped tables
--
--
-- AUTO_INCREMENT for table 'app_user'
--
ALTER TABLE 'app_user'
  MODIFY 'user_id' int(11) NOT NULL AUTO_INCREMENT;

--
```

```

-- AUTO_INCREMENT for table 'django_migrations'
--
ALTER TABLE 'django_migrations'
  MODIFY 'id' int(11) NOT NULL AUTO_INCREMENT;

--
-- AUTO_INCREMENT for table 'game'
--
ALTER TABLE 'game'
  MODIFY 'game_id' int(11) NOT NULL AUTO_INCREMENT;

--
-- Constraints for dumped tables
--

--
-- Constraints for table 'admin_user'
--
ALTER TABLE 'admin_user'
  ADD CONSTRAINT 'fk_admin_user' FOREIGN KEY ('user_id')
    REFERENCES 'app_user' ('user_id');

--
-- Constraints for table 'game'
--
ALTER TABLE 'game'
  ADD CONSTRAINT 'fk_game_outcome' FOREIGN KEY ('outcome')
    REFERENCES 'player' ('user_id'),
  ADD CONSTRAINT 'fk_game_p1' FOREIGN KEY ('player1') REFERENCES
    'player' ('user_id'),
  ADD CONSTRAINT 'fk_game_p2' FOREIGN KEY ('player2') REFERENCES
    'player' ('user_id');

--
-- Constraints for table 'player'
--
ALTER TABLE 'player'
  ADD CONSTRAINT 'fk_player_user' FOREIGN KEY ('user_id')
    REFERENCES 'app_user' ('user_id');
COMMIT;

```

.2 Rating algorithms model

The code for the rating algorithms (their default values, their update methods) is all stored in the same file, *Player models.py*. The code for glicko is heavily inspired by the Glicko-2 Python implementation found on glicko.net, M. Glickman's official website [7].

```

from django.db import models
import math
from django.http import JsonResponse
from user.models import AppUser

```

```
from decimal import Decimal

# Create your models here.
class Player(models.Model):
    # Link to AppUser (one-to-one relationship)
    user = models.OneToOneField(
        AppUser,
        on_delete=models.CASCADE, # If user deleted, delete
        player too
        primary_key=True,
        related_name='player_profile',
        db_column='user_id',
    )

    # Elo ranking
    elo = models.DecimalField(
        max_digits=6,
        decimal_places=2,
        default=1500.00,
        help_text="Elo rating"
    )

    # Glicko-2 ranking
    glicko = models.DecimalField(
        max_digits=6,
        decimal_places=2,
        default=1500.00,
        help_text="Glicko rating"
    )

    glicko_rd = models.DecimalField(
        max_digits=6,
        decimal_places=2,
        default=350.00,
        help_text="Rating Deviation"
    )

    volatility = models.DecimalField(
        max_digits=10,
        decimal_places=8,
        default=0.06,
        help_text="Volatility"
    )

    # Game statistics
    tries = models.IntegerField(
        default=0,
        help_text="Total number of games played"
    )

    last_game_date = models.DateTimeField(
```

```

        null=True,
        blank=True,
        help_text="Date of last game played"
    )

    gamecount=models.IntegerField(
        default=0,
        help_text="Total number of games played"
    )

    _tau = 0.5

    class Meta:
        db_table = 'player'

    def __str__(self):
        return f"Player: {self.user.name} (Elo: {self.elo},
            Glicko: {self.glicko})"

    #Rating methods
    #Update elo ranking
    def update_elo(self, exp, result): #Arguments: exp=expected
        score, calculated in "games" class; result=actual score
        (1=win, 0=loss, 0.5=draw)
        #Use FIDE rating for k, found in https://ratings.fide.com
        /calc.phtml?page=change

        exp = Decimal(str(exp))
        result = Decimal(str(result))

        if self.gamecount < 8 : #placement period of 8 games
            k=Decimal('40')
        elif self.elo < 2400:
            k=Decimal('20')
        elif self.elo>=2400:
            k=Decimal('10')
        #New ranking
        rprime=self.elo + k*(result-exp)
        self.elo = rprime
        self.gamecount+=1
        return self.elo

    #Update glicko ranking
    #These methods are a tweaked version of the algorithm found
    in https://github.com/ryankirkman/pyglicko2/blob/master/
    glicko2.py.
    #Rating formulas are updated to handle only one opponent at a
    time, providing real-time updates.

    def _v(self, opp_rating, opp_RD):
        # expected variance of the players rating based on

```

```

        game_outcomes
tempSum=0
tempE=self._E(opp_rating, opp_RD)
tempSum= math.pow(self._g(opp_RD), 2) * tempE * (1 -
    tempE)
return 1 / tempSum

def _E(self, opp_rating, opp_rd):
    # expected score against an opponent
    #Convert to glicko scale:
    mu=self.rating(self.glicko)
    opp_rating=self.rating(opp_rating)
    #Compute expected score:
    return 1 / (1 + math.exp(-1 * self._g(opp_rd) * \
        (mu - opp_rating)))

def _g(self, opp_rd):
    #The Glicko2 g(RD) function.
    opp_rd=self.phi(opp_rd)
    return 1/math.sqrt(1 + 3 * math.pow(opp_rd, 2) / math.pow
        (math.pi, 2))

def preRD(self):
    phi= self.phi(float(self.glicko_rd))
    sigma=float(self.volatility)
    phiprime = math.sqrt(math.pow(phi, 2) + math.pow(sigma,
        2)) #Compute new RD
    self.glicko_rd = Decimal(str(hiprime * 173.7178)) #
        convert back to original scale

def update_glicko(self, opp_rating, opp_RD, outcome):
    #Calculates the new rating and rating deviation of the
        player.

    #Convert variables
    opp_rating=float(opp_rating)
    opp_RD=float(opp_RD)
    outcome=float(outcome)

    v=self._v(opp_rating, opp_RD)
    newvol=self.newvol(opp_rating, opp_RD, outcome, v)
    self.volatility=Decimal(str(newvol))
    self.preRD()

    phi = 1/math.sqrt((1 / math.pow(self.phi(float(self.
        glicko_rd)), 2))+ 1/v)

    mu=self.rating(float(self.glicko))
    tempsum=self._g(opp_RD) * (outcome - self._E(opp_rating,
        opp_RD))
    mu+= math.pow(phi, 2) * tempsum

```

```

        self.glicko_rd = Decimal(str(phi * 173.7178))
        self.glicko = Decimal(str(mu * 173.7178 + 1500))

    return self.glicko

def newvol(self, opp_rating, opp_RD, outcome, v):
    #Calculates the new volatility of the player.
    i = 0
    delta = self._delta(opp_rating, opp_RD, outcome, v)
    a = math.log(math.pow(self.volatility, 2))
    tau = self._tau
    x0 = a
    x1 = 0

    while x0 != x1:
        # New iteration, so x(i) becomes x(i-1)
        x0 = x1
        d = math.pow(self.rating(self.glicko), 2) + v + math.exp(x0)
        h1 = -(x0 - a) / math.pow(tau, 2) - 0.5 * math.exp(x0) \
            / d + 0.5 * math.exp(x0) * math.pow(delta / d, 2)
        h2 = -1 / math.pow(tau, 2) - 0.5 * math.exp(x0) * \
            (math.pow(self.rating(self.glicko), 2) + v) \
            / math.pow(d, 2) + 0.5 * math.pow(delta, 2) * math.exp(x0) \
            * (math.pow(self.rating(self.glicko), 2) + v - math.exp(x0)) / math.pow(d, 3)
        x1 = x0 - (h1 / h2)

    return math.exp(x1 / 2)

def _delta(self, opp_rating, opp_RD, outcome, v):
    #the estimated improvement in rating by comparing the pre
    #period rating to the performance rating based only on
    #game outcomes
    sum=self._g(opp_RD) * (outcome - self._E(opp_rating,
        opp_RD))
    return v*sum

#helpers for glicko scale:
def rating(self, r):
    return float((r- 1500)) / 173.7178
def phi(self, RD):
    return float(RD / 173.7178)

class Meta:
    db_table = 'player'
    managed = False # DB handled in xampp

```


.3 Test code

.3.1 Rating algorithms test code

The rating algorithms are simply copy pasted from the main implementation. The test cases are generated by AI. The test method and results are explained in Chapter 6.

```
import math
from decimal import Decimal

#MODELS
TAU = 0.5

class TempPlayer:
    def __init__(self, name, elo=1500.00, glicko=1500.00,
        glicko_rd=350.00, volatility=0.06):
        self.name = name
        self.elo = Decimal(str(elo))
        self.glicko = Decimal(str(glicko))
        self.glicko_rd = Decimal(str(glicko_rd))
        self.volatility = Decimal(str(volatility))
        self.gamecount = 0
        self.wins = 0
        self.losses = 0
        self._tau = TAU

    def rating(self, r):
        return float((r - 1500)) / 173.7178

    def phi(self, RD):
        return float(RD / 173.7178)

#ELO
def update_elo(self, exp, result):
    exp = Decimal(str(exp))
    result = Decimal(str(result))

    if self.gamecount < 8:
        k = Decimal('40')
    elif self.elo < 2400:
        k = Decimal('20')
    elif self.elo >= 2400:
        k = Decimal('10')

    #New ranking
    rprime=self.elo + k*(result-exp)
    self.elo = rprime
    self.gamecount+=1
    return self.elo

#Glicko
def _v(self, opp_rating, opp_RD):
    # expected variance of the players rating based on
```

```

        game_outcomes
tempSum=0
tempE=self._E(opp_rating, opp_RD)
tempSum= math.pow(self._g(opp_RD), 2) * tempE * (1 -
    tempE)
return 1 / tempSum

def _E(self, opp_rating, opp_rd):
    # expected score against an opponent
    #Convert to glicko scale:
    mu=self.rating(self.glicko)
    opp_rating=self.rating(opp_rating)
    #Compute expected score:
    return 1 / (1 + math.exp(-1 * self._g(opp_rd) * \
        (mu - opp_rating)))

def _g(self, opp_rd):
    #The Glicko2 g(RD) function.
    opp_rd=self.phi(opp_rd)
    return 1/math.sqrt(1 + 3 * math.pow(opp_rd, 2) / math.pow
        (math.pi, 2))

def preRD(self):
    phi= self.phi(float(self.glicko_rd))
    sigma=float(self.volatility)
    phiprime = math.sqrt(math.pow(phi, 2) + math.pow(sigma,
        2)) #Compute new RD
    self.glicko_rd = Decimal(str(hiprime * 173.7178)) #
        convert back to original scale

def update_glicko(self, opp_rating, opp_RD, outcome):
    #Calculates the new rating and rating deviation of the
        player.

    #Convert variables
    opp_rating=float(opp_rating)
    opp_RD=float(opp_RD)
    outcome=float(outcome)

    v=self._v(opp_rating, opp_RD)
    newvol=self.newvol(opp_rating, opp_RD, outcome, v)
    self.volatility=Decimal(str(newvol))
    self.preRD()

    phi = 1/math.sqrt((1 / math.pow(self.phi(float(self.
        glicko_rd)), 2))+ 1/v)

    mu=self.rating(float(self.glicko))
    tempsum=self._g(opp_RD) * (outcome - self._E(opp_rating,
        opp_RD))
    mu+= math.pow(phi, 2) * tempsum

```

```

        self.glicko_rd = Decimal(str(phi * 173.7178))
        self.glicko = Decimal(str(mu * 173.7178 + 1500))

    return self.glicko

def newvol(self, opp_rating, opp_RD, outcome, v):
    #Calculates the new volatility of the player.
    i = 0
    delta = self._delta(opp_rating, opp_RD, outcome, v)
    a = math.log(math.pow(self.volatility, 2))
    tau = self._tau
    x0 = a
    x1 = 0

    while x0 != x1:
        # New iteration, so x(i) becomes x(i-1)
        x0 = x1
        d = math.pow(self.rating(self.glicko), 2) + v + math.exp(x0)
        h1 = -(x0 - a) / math.pow(tau, 2) - 0.5 * math.exp(x0) \
            / d + 0.5 * math.exp(x0) * math.pow(delta / d, 2)
        h2 = -1 / math.pow(tau, 2) - 0.5 * math.exp(x0) * \
            (math.pow(self.rating(self.glicko), 2) + v) \
            / math.pow(d, 2) + 0.5 * math.pow(delta, 2) * math.exp(x0) \
            * (math.pow(self.rating(self.glicko), 2) + v - math.exp(x0)) / math.pow(d, 3)
        x1 = x0 - (h1 / h2)

    return math.exp(x1 / 2)

def _delta(self, opp_rating, opp_RD, outcome, v):
    #the estimated improvement in rating by comparing the pre
    #period rating to the performance rating based only on
    #game outcomes
    sum=self._g(opp_RD) * (outcome - self._E(opp_rating,
        opp_RD))
    return v*sum

#helpers for glicko scale:
def rating(self, r):
    return float((r- 1500)) / 173.7178
def phi(self, RD):
    return float(RD / 173.7178)

#####

#Test case

def elo_expected(rating_a, rating_b):

```

```

    return 1 / (1 + 10 ** ((float(rating_b) - float(rating_a)) /
        400))

def play_match(p1, p2, winner):
    if winner == 1:
        p1_result, p2_result = 1, 0
        p1.wins += 1
        p2.losses += 1
    elif winner == 2:
        p1_result, p2_result = 0, 1
        p2.wins += 1
        p1.losses += 1
    else:
        p1_result, p2_result = 0.5, 0.5

    # Elo
    exp1 = elo_expected(p1.elo, p2.elo)
    exp2 = elo_expected(p2.elo, p1.elo)
    p1.update_elo(exp1, p1_result)
    p2.update_elo(exp2, p2_result)

    # Glicko - save opponent values before updating
    opp_rating = float(p2.glicko)
    opp_rd = float(p2.glicko_rd)
    opp_rating2 = float(p1.glicko)
    opp_rd2 = float(p1.glicko_rd)

    p1.update_glicko(opp_rating, opp_rd, p1_result)
    p2.update_glicko(opp_rating2, opp_rd2, p2_result)

def print_player(p):
    print(f" {p.name:>8} | Elo: {float(p.elo):7.2f} | "
          f"Glicko: {float(p.glicko):7.2f} | "
          f"RD: {float(p.glicko_rd):6.2f} | "
          f"Vol: {float(p.volatility):.6f} | "
          f"W/L: {p.wins}/{p.losses}")

#1st test: equal players
def test_1_equal_players():
    print("\n" + "=" * 60)
    print("TEST 1: Two equal players, alternating wins")
    print("=" * 60)

    a = TempPlayer("Alice")
    b = TempPlayer("Bob")

    for i in range(20):
        winner = 1 if i % 2 == 0 else 2
        play_match(a, b, winner)

```

```

print_player(a)
print_player(b)

# Both should stay close to 1500
assert abs(float(a.elo) - 1500) < 50, f"FAIL: Alice Elo
    drifted to {a.elo}"
assert abs(float(b.elo) - 1500) < 50, f"FAIL: Bob Elo drifted
    to {b.elo}"
assert abs(float(a.glicko) - 1500) < 100, f"FAIL: Alice
    Glicko drifted to {a.glicko}"
assert abs(float(b.glicko) - 1500) < 100, f"FAIL: Bob Glicko
    drifted to {b.glicko}"
print("PASSED: Equal players stay near 1500")

#2nd test: one player dominates
def test_2_dominant_player():
    print("\n" + "=" * 60)
    print("TEST 2: Player A always beats Player B")
    print("=" * 60)

    a = TempPlayer("Strong")
    b = TempPlayer("Weak")

    for _ in range(15):
        play_match(a, b, winner=1)

    print_player(a)
    print_player(b)

    assert float(a.elo) > float(b.elo), "FAIL: Winner should have
        higher Elo"
    assert float(a.glicko) > float(b.glicko), "FAIL: Winner
        should have higher Glicko"
    assert float(a.elo) > 1500, "FAIL: Winner Elo should be above
        1500"
    assert float(b.elo) < 1500, "FAIL: Loser Elo should be below
        1500"
    print("PASSED: Dominant player rated higher")

#3rd test: new player vs established player: upset from the new
player
def test_3_upset():
    print("\n" + "=" * 60)
    print("TEST 3: Low-rated player beats high-rated player")
    print("=" * 60)

    high = TempPlayer("Favorite", elo=1800, glicko=1800,
        glicko_rd=100)
    low = TempPlayer("Underdog", elo=1200, glicko=1200, glicko_rd
        =100)

```

```

elo_before = float(high.elo)
glicko_before = float(high.glicko)

play_match(high, low, winner=2) # Upset!

elo_drop = elo_before - float(high.elo)
glicko_drop = glicko_before - float(high.glicko)

print_player(high)
print_player(low)
print(f" Favorite Elo drop: {elo_drop:.2f}")
print(f" Favorite Glicko drop: {glicko_drop:.2f}")

assert elo_drop > 10, f"FAIL: Upset should cause significant
    Elo drop, got {elo_drop:.2f}"
assert glicko_drop > 5, f"FAIL: Upset should cause
    significant Glicko drop, got {glicko_drop:.2f}"
print("PASSED: Upset causes meaningful rating change")

#Testing RD decrease over time with games
def test_4_rd_decreases():
    print("\n" + "=" * 60)
    print("TEST 4: Rating Deviation decreases over games")
    print("=" * 60)

    a = TempPlayer("Active")
    b = TempPlayer("Opponent")

    rd_values = [float(a.glicko_rd)]

    for i in range(10):
        winner = 1 if i % 2 == 0 else 2
        play_match(a, b, winner)
        rd_values.append(float(a.glicko_rd))

    print(f" RD progression: {' '.join(f'{rd:.1f}' for rd in
        rd_values)}")

    assert rd_values[-1] < rd_values[0], \
        f"FAIL: RD should decrease from {rd_values[0]:.2f}, got {
            rd_values[-1]:.2f}"
    print("PASSED: RD decreases with more games")

#K-factor adjustment test
def test_6_k_factor():
    print("\n" + "=" * 60)
    print("TEST 6: K-factor is higher during placement (first 8
        games)")
    print("=" * 60)

    a = TempPlayer("NewPlayer")

```

```

b = TempPlayer("Opponent", glicko_rd=100)
b.gamecount = 20 # Established player

# Track Elo changes during placement
placement_changes = []
for i in range(8):
    elo_before = float(a.elo)
    play_match(a, b, winner=1)
    placement_changes.append(float(a.elo) - elo_before)

# Track Elo changes after placement
post_changes = []
for i in range(4):
    elo_before = float(a.elo)
    play_match(a, b, winner=1)
    post_changes.append(float(a.elo) - elo_before)

avg_placement = sum(placement_changes) / len(
    placement_changes)
avg_post = sum(post_changes) / len(post_changes)

print(f" Avg Elo change during placement (k=40): {
    avg_placement:.2f}")
print(f" Avg Elo change after placement (k=20): {avg_post
    :.2f}")

assert avg_placement > avg_post, \
    "FAIL: Placement games should have larger rating changes"
print("PASSED: K-factor correctly higher during placement")

#Final test: competition
def test_7_tournament():
    print("\n" + "=" * 60)
    print("TEST 7: Tournament with known skill ordering")
    print("=" * 60)

    import random
    random.seed(123)

    # True skill: Best > Good > Average > Weak
    # Higher skill = higher win probability
    players = [
        TempPlayer("Best"),
        TempPlayer("Good"),
        TempPlayer("Average"),
        TempPlayer("Weak"),
    ]
    true_skill = [0.85, 0.65, 0.45, 0.25]

    for _ in range(100):
        i, j = random.sample(range(4), 2)

```

```

        # Probability i beats j based on skill difference
        prob_i_wins = true_skill[i] / (true_skill[i] + true_skill[j])
        winner = 1 if random.random() < prob_i_wins else 2
        play_match(players[i], players[j], winner)

print("  Final standings (should be Best > Good > Average >
      Weak):")
for p in players:
    print_player(p)

elos = [float(p.elo) for p in players]
glickos = [float(p.glicko) for p in players]

# Check ordering
elo_ordered = all(elos[i] >= elos[i + 1] for i in range(len(
    elos) - 1))
glicko_ordered = all(glickos[i] >= glickos[i + 1] for i in
    range(len(glickos) - 1))

if elo_ordered:
    print("      PASSED: Elo ranking matches true skill order")
else:
    print(f"      WARNING: Elo order is {[p.name for p in
        sorted(players, key=lambda x: -float(x.elo))]}")

if glicko_ordered:
    print("      PASSED: Glicko ranking matches true skill
        order")
else:
    print(f"      WARNING: Glicko order is {[p.name for p
        in sorted(players, key=lambda x: -float(x.glicko))]}")

if __name__ == "__main__":
    print("      RATING ALGORITHM TEST SUITE")
    print("No database. No Django. Pure math.\n")

    test_1_equal_players()
    test_2_dominant_player()
    test_3_upset()
    test_4_rd_decreases()
    test_6_k_factor()
    test_7_tournament()

    print("\n" + "=" * 60)
    print("ALL TESTS COMPLETE")
    print("=" * 60)

```